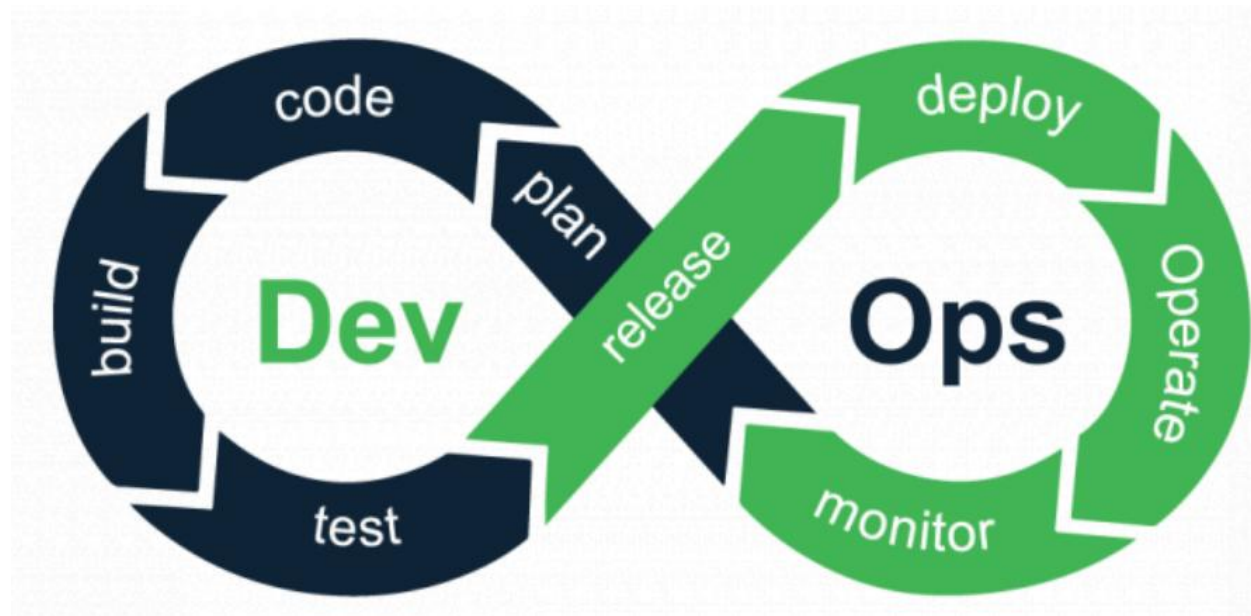




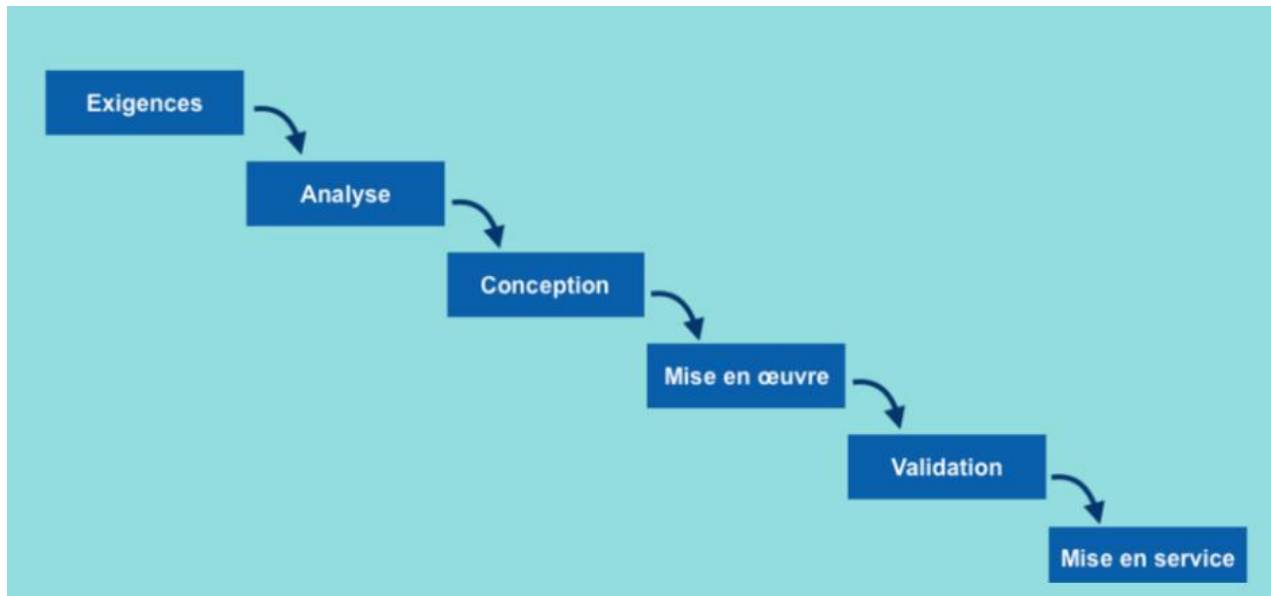
Chapitre 1

Introduction au DevOps

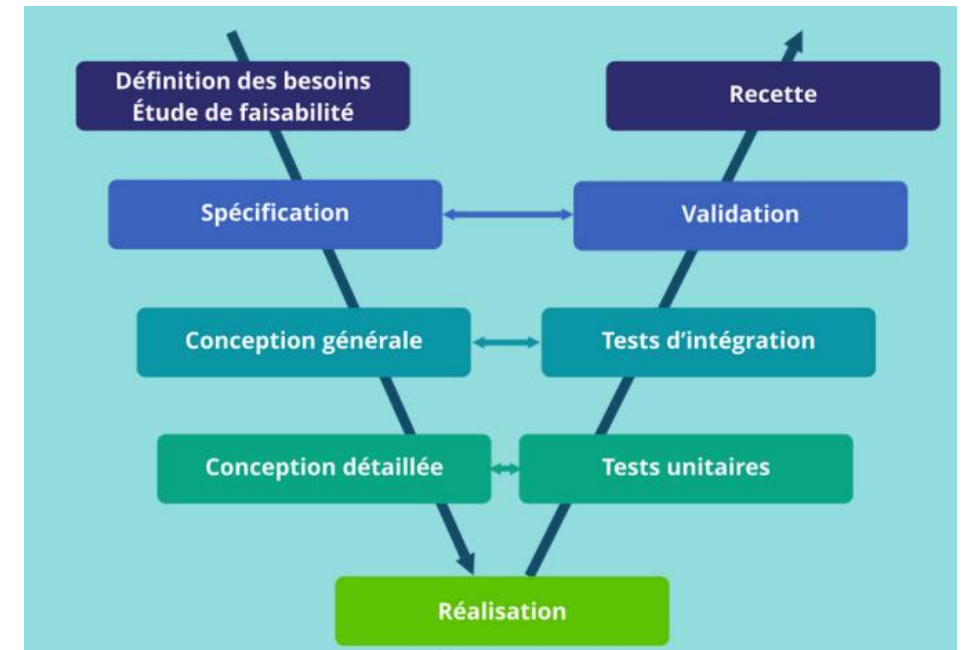


Effet Tunnel

- Au début des années 2000, les méthodes de développement traditionnelles, comme le cycle en V, souffraient d'un problème majeur : l'effet tunnel. Pendant des mois, les développeurs travaillaient sur un produit sans retour concret des clients, aboutissant souvent à des livrables qui ne correspondaient pas aux attentes réelles.



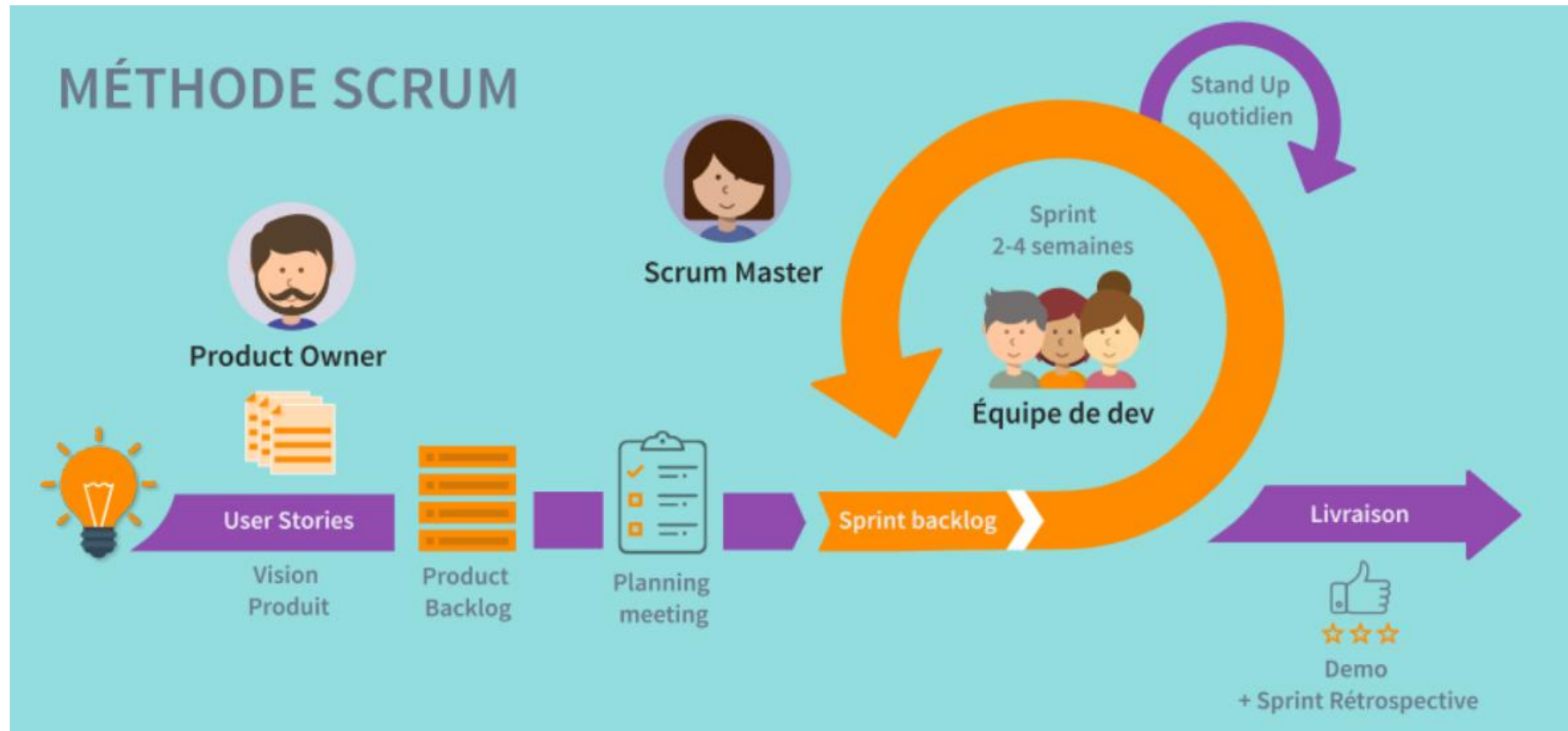
Modèle en cascade



Modèle en V

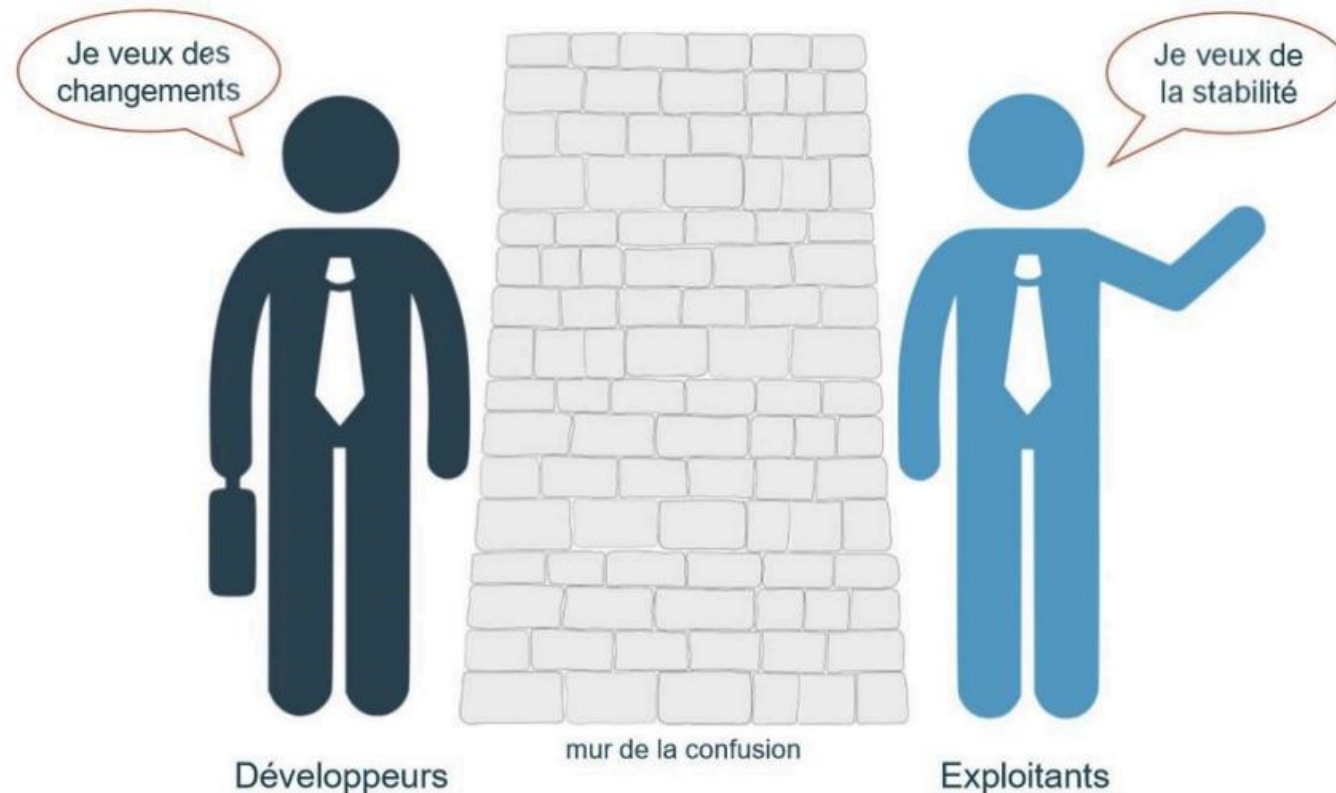
Le Mouvement Agile

- Pour répondre à cette problématique, le mouvement Agile est apparu, notamment avec des méthodes comme **Scrum**, apportant des cycles de développement courts (sprints), des revues régulières et une meilleure collaboration entre les équipes métier et techniques.



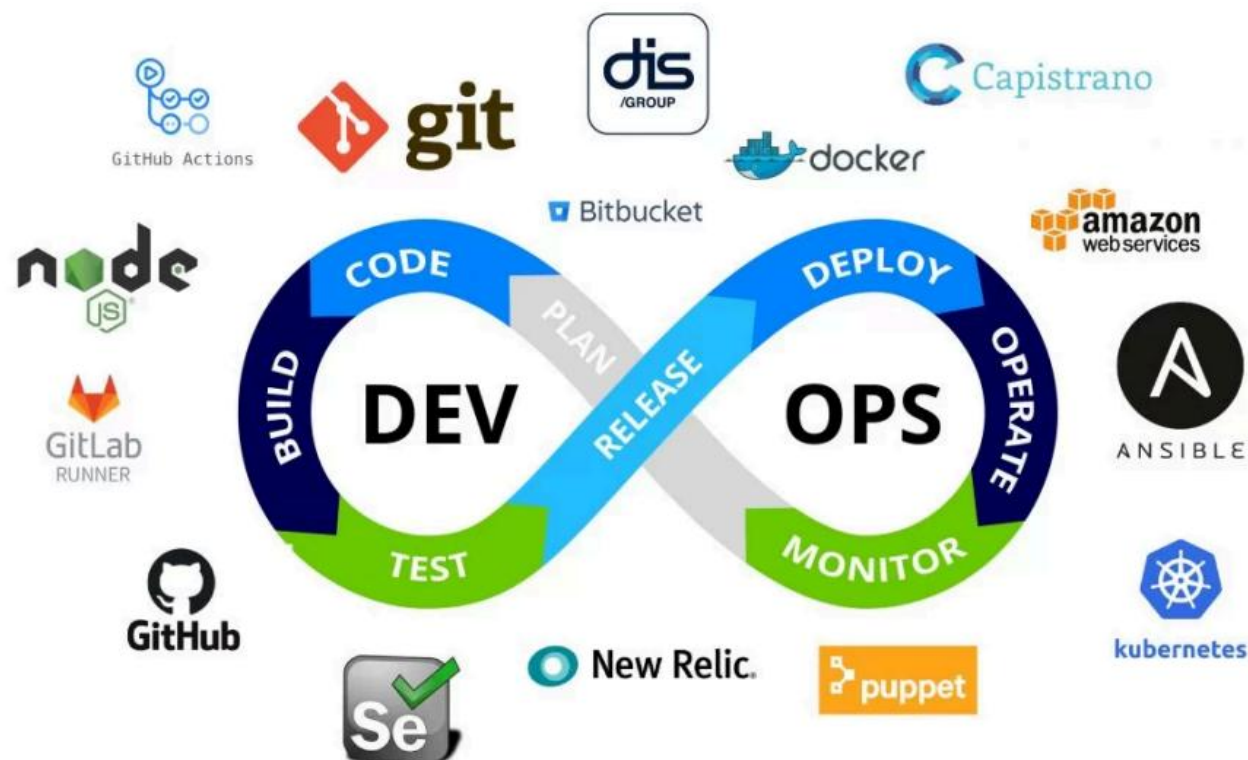
De l'Agile vers le DevOps

- Cependant, même avec Agile, un fossé subsistait entre le développement (qui livrait rapidement des fonctionnalités) et les opérations (chargées du déploiement en production, souvent plus lentes pour garantir la stabilité). Ce désalignement ralentissait encore l'innovation.



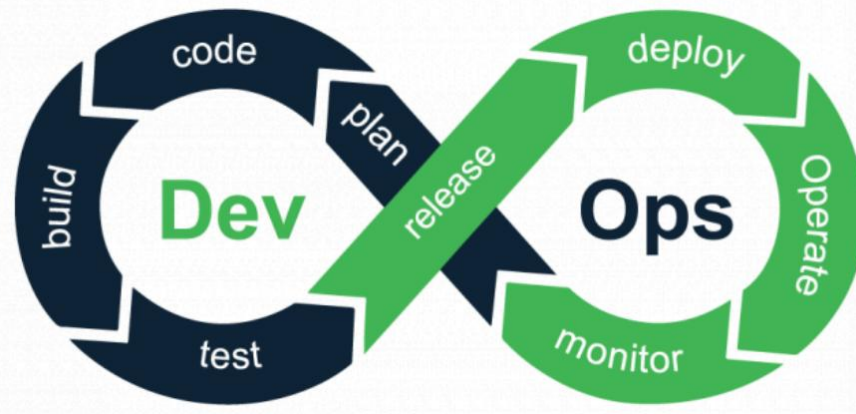
La culture DevOps

- C'est dans ce contexte qu'est né DevOps : DevOps vise à aligner développement et opérations, en favorisant l'automatisation, la collaboration, l'intégration continue et le déploiement continu. L'objectif est clair : livrer plus rapidement des logiciels de qualité, tout en assurant leur fiabilité en production.

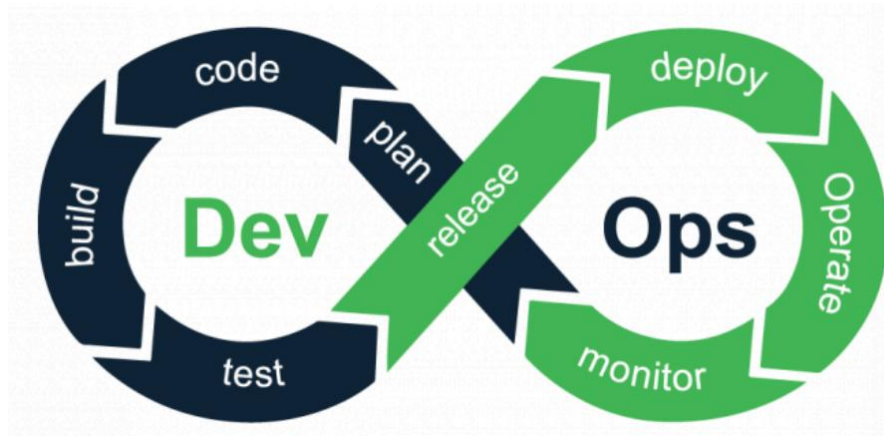


Modèle Cascade / Modèle V vs Agile / DevOps

Critère	Modèle Cascade	Modèle en V	Agile	DevOps
Gestion des changements	Difficile, coûteuse	Difficile	Facile et continue	Facile et automatisée
Livraison	Une seule livraison finale	Une seule livraison mais mieux testée	Livraisons fréquentes	Livraisons continues (CI/CD)
Feedback du client	Tardif (fin du projet)	Tardif mais validation par phase	Continu à chaque sprint	Continu via monitoring et releases
Collaboration des équipes	Équipes en silos	Silos mais synchronisation tests → dev	Collaboration forte	Collaboration Dev + Ops + QA



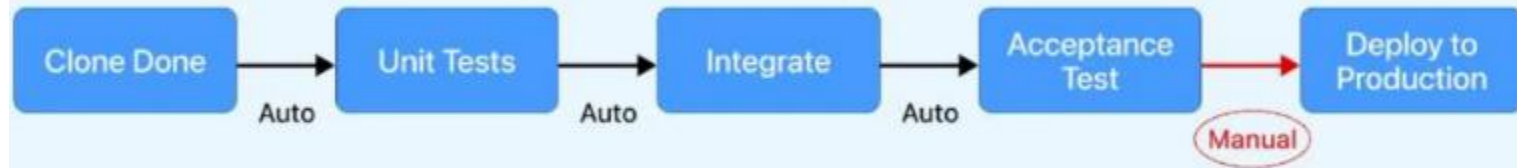
- **Plan:** définir les besoins, fonctionnalités, tâches et roadmap du projet
outils: Jira, Projets github.
- **Code:** écrire le code source du projet
outils: Java, C, C++...
- **Build:** générer les fichiers exécutables(artifact) c'est-à-dire packager l'application avec ses dépendances.
outils: maven, gradle...
- **Test:** valider le code automatiquement pour détecter les erreurs.
outils: Junit, Sélénium...
- **Release:** approuver et préparer les artefacts pour la production.
outils: artifactory, nexus...



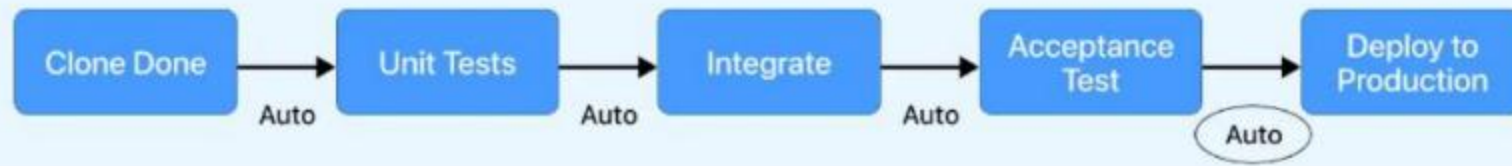
- **Deploy:** déployer automatiquement l'application dans un environnement de Dev, Préprod, Prod
outils: GitHub Actions, docker, ArgoCD, Kubernetes, OpenShift.
- **Operate:** faire fonctionner l'application en production et gérer l'infrastructure.
outils: Kubernetes, OpenShift, Docker, Terraform, Ansible.
- **Monitor:** collecter des métriques, logs et alertes pour garantir le bon fonctionnement du système.
outils: Prometheus, Grafana, Loki, Tempo

Continuous Delivery vs Continuous Deployment

Continuous Delivery

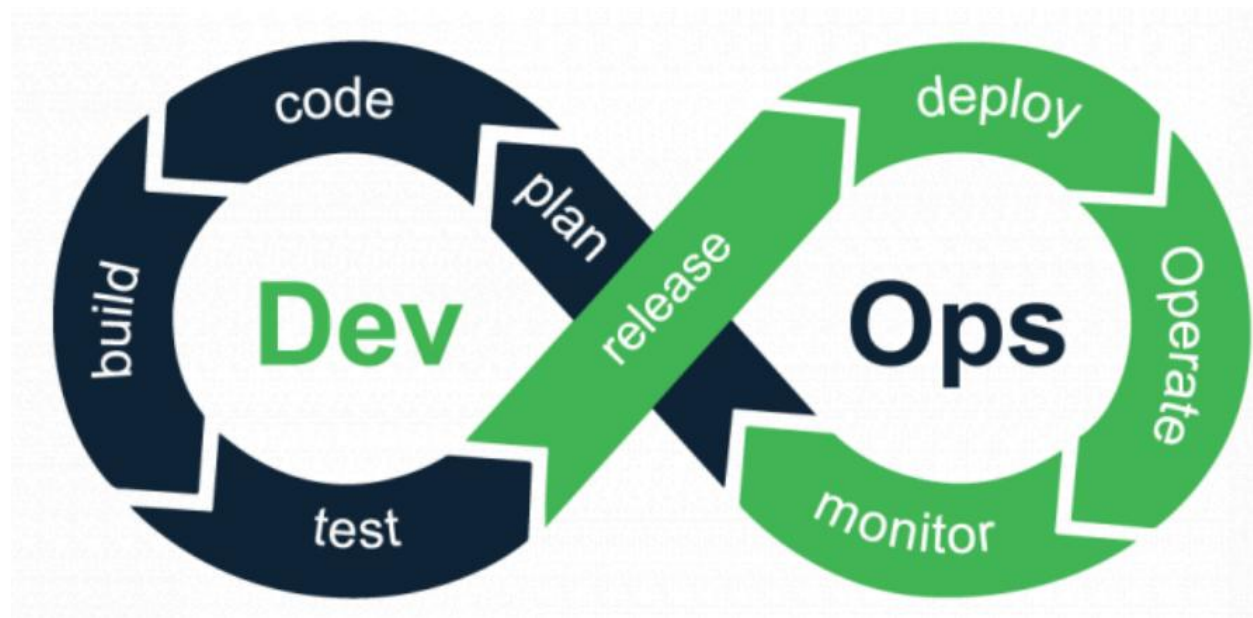


Continuous Deployment



Site Reliability Engineering

- **Incident Management** : Restaurer rapidement le service après une panne ou anomalie pour minimiser l'impact sur l'utilisateur.
- **Problem Management** : Identifier la cause profonde des incidents récurrents (RCA) et mettre en place des actions pour éviter qu'ils ne se reproduisent.
- **Change Management** : Valider les solutions et planifier les déploiements, mises en production afin de réduire les risques et assurer la stabilité.



Chapitre 2

La gestion de contrôle de version (GIT)

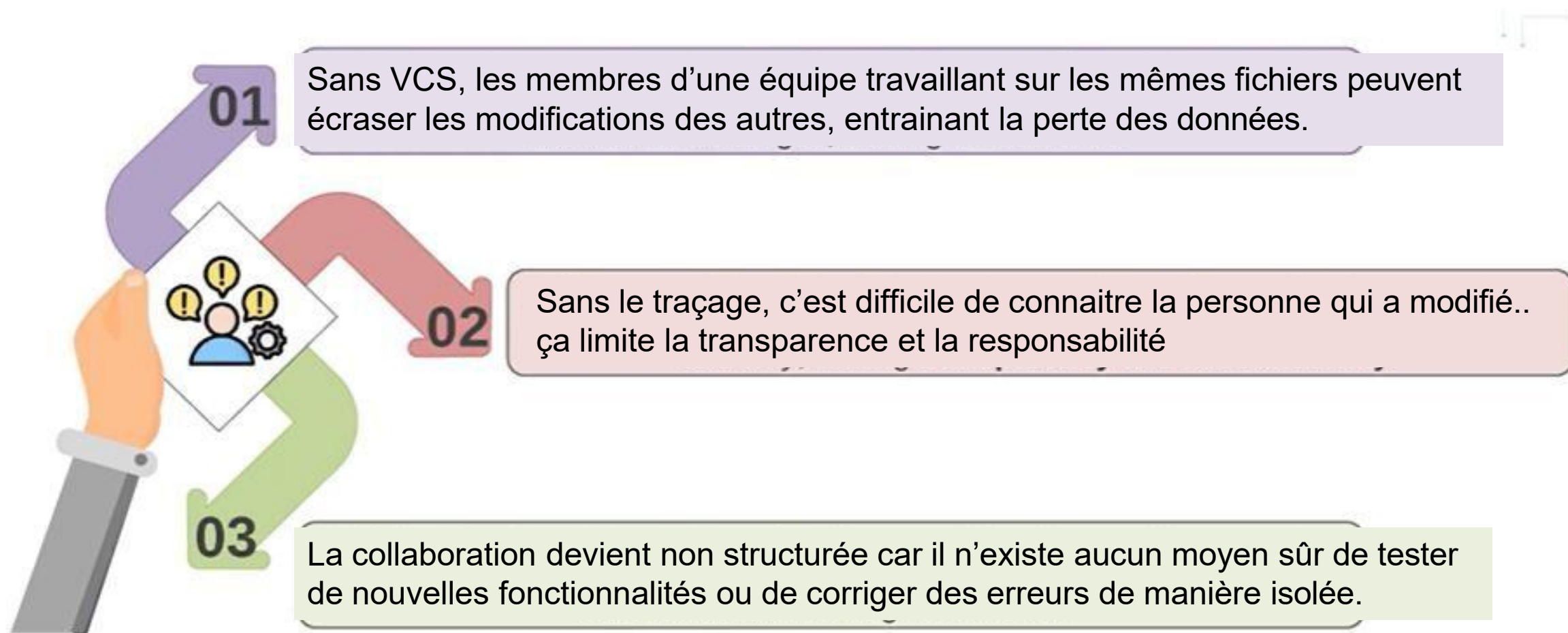


Plan

1. L'importance du contrôle de versions
2. Présentation d'un système de contrôle de versions
3. GIT introduction
4. Création d'un repo sur github
5. Les commandes de base git
6. Les branches et la gestion des conflits
7. Le pull request
8. Commandes git avancées

1. L'importance de contrôle de versions

Motivation



2. Présentation d'un système de contrôle de versions

C'est quoi le contrôle de version ?

- Un système de contrôle de version, ou VCS (Version Control System), est un outil logiciel qui permet de suivre et de gérer les modifications apportées à un ensemble de fichiers, généralement du code source, au fil du temps.
- Il permet aux équipes de développement de collaborer efficacement, de revenir à des versions antérieures en cas d'erreur et de gérer les différentes versions d'un projet.



Système de contrôle de version - outils



3. GIT introduction

GIT c'est quoi

- **Git** est un système de contrôle de version open source, distribué, gratuit et très performant, conçu pour gérer les modifications apportées aux fichiers de projets, des plus petits aux plus grands. Il permet aux développeurs de suivre l'historique des modifications, de collaborer efficacement et de revenir à des versions antérieures si nécessaire.
- **Git** a été fondé par **Linus Torvalds**, le créateur du noyau Linux, en 2005.

- ✓ Facilite le contrôle de version et le traçage de l'historique
- ✓ Améliore l'intégrité du code et réduit les conflits
- ✓ Permet le branchement et la fusion pour un travail indépendant

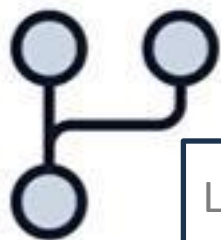


Github le dépôt distant



- **GitHub** est une plateforme web et un service cloud qui sert d'hébergement et de gestion pour les projets de développement de logiciels, en utilisant le système de contrôle de version Git.
- Elle permet à des millions de développeurs de travailler ensemble sur des projets, de suivre les modifications du code, de partager leur travail et de gérer le cycle de développement grâce à des fonctionnalités comme le suivi des bugs, les demandes de modification.
- Autres exemples de dépôts distants : **BITBUCKET ET GITLAB**

Les composants de GIT



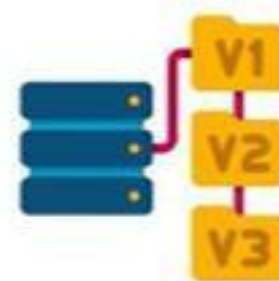
Branch

Les modifications apportées sur une branche sont isolées, évitant de "polluer" la branche principale (comme main ou master) avec du code en cours ou instable.



Commit

Il sert à enregistrer un instantané des modifications staging de votre projet dans son historique local



Tags

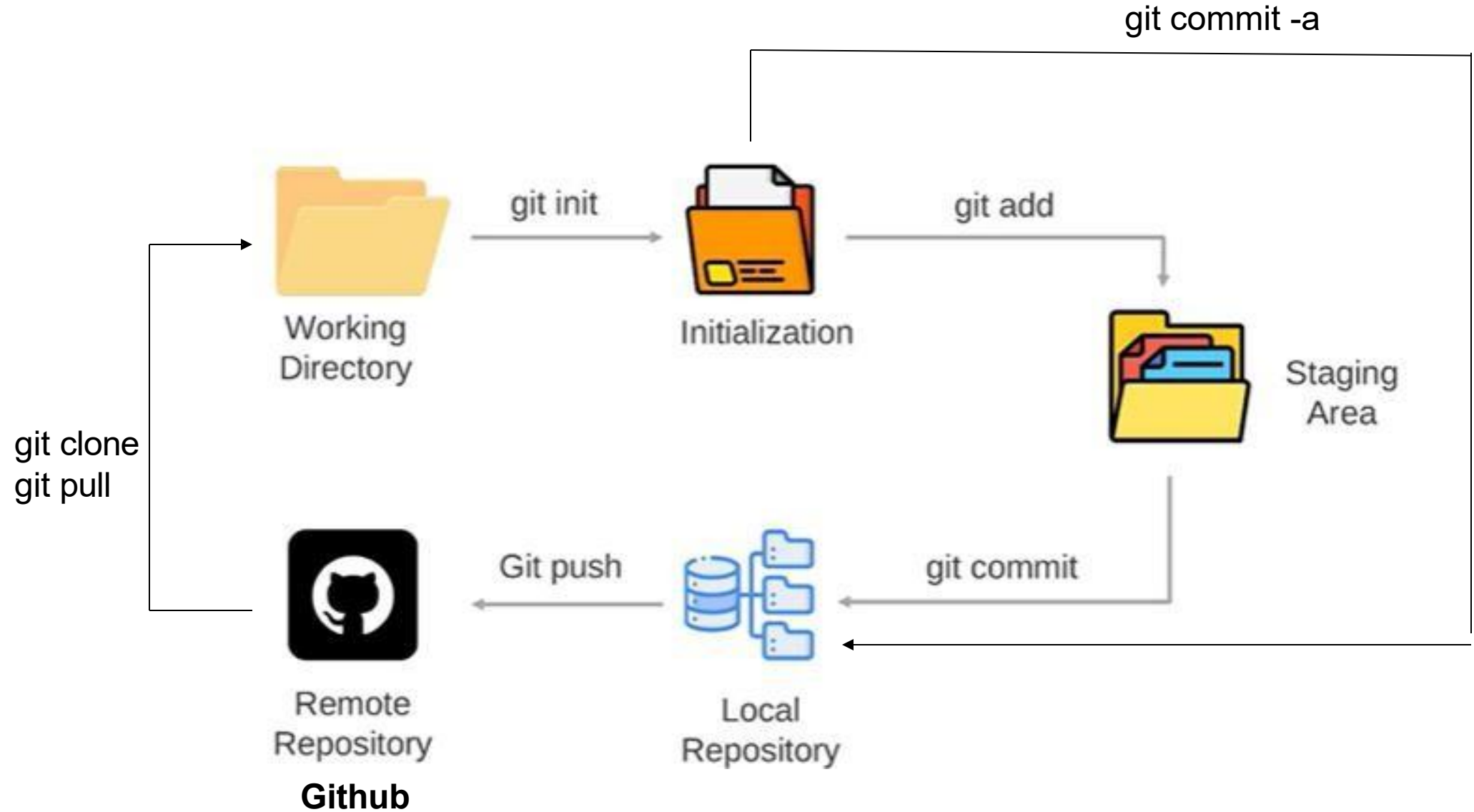
Une étiquette (un nom) qui pointe vers un point fixe de l'historique de votre projet Git, généralement un commit



HEAD

est une référence symbolique qui pointe vers le dernier commit de la branche sur laquelle vous travaillez actuellement

Comment fonctionne GIT ?



Installation de WSL: Windows Subsystem for Linux (distribution Ubuntu)

- Ouvrir l'invite de commandes en tant qu'administrateur
- Installer WSL: `wsl --install`
- Lancer WSL: `wsl`

Installation de GIT sous WSL

- Lancer les mises à jours de WSL avant d'installer GIT: `sudo apt-get update`
- Installer GIT: `sudo apt-get install git -y`
- Vérifier que GIT est bien installé: `git --version`

Configuration de GIT sous WSL

- Vérifier que GIT est correctement installé: `git --version`
- Définissez votre nom d'utilisateur et votre adresse e-mail avec la commande `git config` pour éviter d'être invité à le faire à chaque action :
`git config --global user.name «name »`
`git config --global user.email «email »`
- Afficher toutes les configurations Git de votre système: `git config --list`

Génération du token sur github

Pour générer un jeton d'accès personnel (PAT) sur GitHub et l'utiliser pour des opérations de push, suivez ces étapes :

1. Accédez aux paramètres de votre compte GitHub:

Cliquez sur votre photo de profil en haut à droite, puis sélectionnez "Paramètres".

2. Allez dans les paramètres développeur:

Dans la barre latérale gauche, cliquez sur "Paramètres développeur" puis "Jetons d'accès personnels".

3. Générez un nouveau jeton:

Cliquez sur "Générer un nouveau jeton", puis "Générer un nouveau jeton (classique)".

4. Création d'un repo github

Introduction git/github

- Git est un logiciel de versioning créé en 2005 par Linus Torvalds, le créateur de Linux.
- Un logiciel de versioning, ou logiciel de gestion de version est un logiciel qui permet de conserver un historique des modifications effectuées sur un projet afin de pouvoir rapidement identifier les changements effectués et de revenir à une ancienne version en cas de problème.
- Les logiciels de gestion de versions sont quasiment incontournables aujourd'hui car ils facilitent grandement la gestion de projets et car ils permettent de travailler en équipe de manière beaucoup plus efficace.
- Parmi les logiciels de gestion de versions, Git et il est donc indispensable pour tout développeur de savoir utiliser Git.

Introduction git/github

- GitHub est un service en ligne qui permet d'héberger des repos Git.
- Une grande partie des repos hébergés sur GitHub sont publics, c'est-à-dire que n'importe qui peut **cloner** le code de ces repos et contribuer à leur développement en créant de nouvelles fonctionnalités.
- Git est un logiciel de gestion de version tandis que GitHub est un service en ligne d'hébergement de repos Git.

Création d'un repo sur github

Nous allons maintenant créer un repo distant sur GitHub :

- Cliquez sur le “+” en haut à droite de la page, puis “New repository”.
- Attribuez un nom à votre repo. Par défaut, votre repo est public et tout le monde a accès à votre code (droits de lecture seulement). Puis cliquez sur “Create repository”.

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk ().*

1 General

Owner * / Repository name *

 Hichem-Khelil / TP-Git

✔ TP-Git is available.

Great repository names are short and memorable. How about [improved-couscous](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository

Public

Start with a template
Templates pre-configure your repository with files.

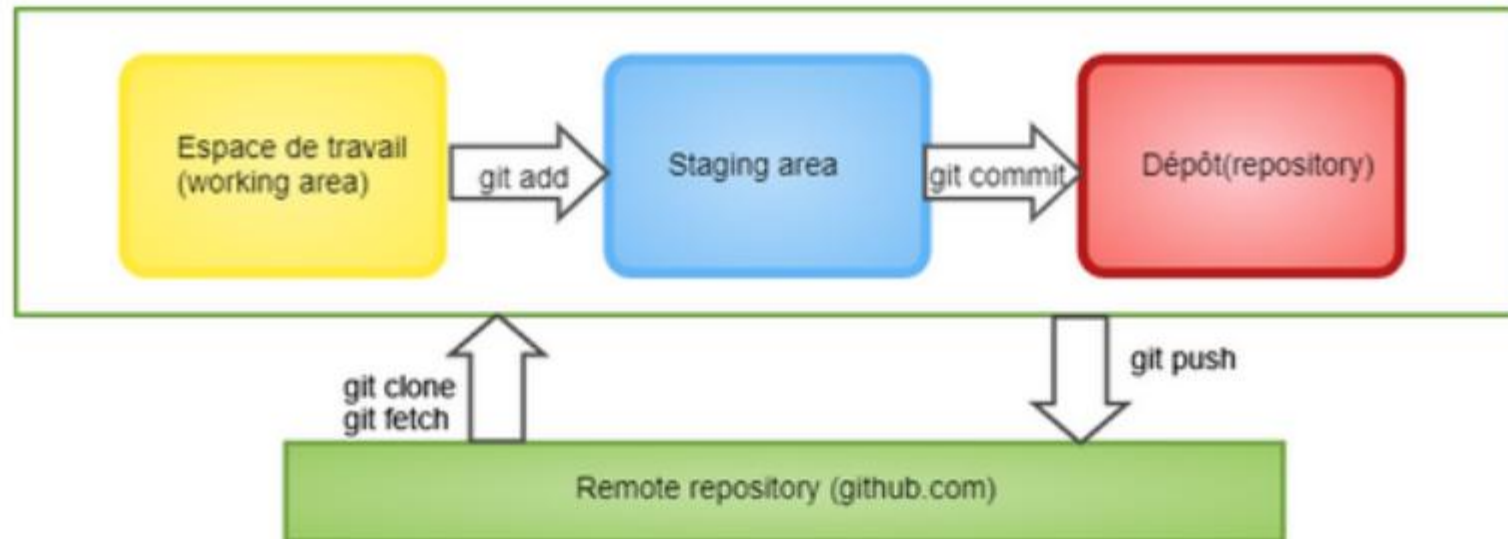
No template

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Off

La structure générale de git

- Working area: l'endroit où tu modifies les fichiers localement.
- Staging area: l'endroit où tu prépares les fichiers avant de créer un commit.
- Repository: l'endroit où sont stockés définitivement les commits et l'historique du projet.



5. Les Commandes de base git

Création d'une copie locale du repo

- **git init:** Permet d'initialiser un repo local vide dans votre répertoire courant.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /mnt/c/devops-module/TP-Git/.git/
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Création d'une copie locale du repo

- **git status:** permet d'afficher l'état actuel du dépôt: fichiers modifiés, les fichiers prêts pour commit(dans la staging area), la branche sur laquelle tu travailles...

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Création d'une copie locale du repo

- Ajouter un fichier java qui permet d'afficher « Hello World »

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ vi HelloWorld.java
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ cat HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    HelloWorld.java

nothing added to commit but untracked files present (use "git add" to track)
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Création d'une copie locale du repo

- **git add:** met les fichiers que tu as modifiés dans le staging area pour les préparer à être inclus dans le prochain commit.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git add .  
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

- **git add . :** le « . » pour dire ajouter tous les fichiers modifiés, créés ou supprimés du dossier courant et de ses sous-dossiers au staging area.

Création d'une copie locale du repo

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git add .
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   HelloWorld.java

hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Création d'une copie locale du repo

- **git commit -m « message »**: Permet de valider ce qui se trouve dans le staging area, en ajoutant un message descriptif pour expliquer le changement.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git commit -m "add java project"
[master (root-commit) f5374ed] add java project
 1 file changed, 5 insertions(+)
 create mode 100644 HelloWorld.java
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```


Création d'une copie locale du repo

- **git branch:** lister les branches existantes.
- **git branch -M main:** changer le nom de la branche par défaut qui est master (devenue main depuis 2020).

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch
* master
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch -M main
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch
* main
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Création d'une copie locale du repo

- **git remote add <nom> <url>:** Permet de lier le repo local (initié par init) à un repo distant (url sur GitHub). Tu dis à Git où envoyer (push) et d'où récupérer (pull) ton code.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git remote add origin https://github.com/Hichem-Khelil/TP-Git.git
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

- **git push -u origin main:** Permet d'envoyer ton code sur GitHub(repo distant défini par origin) et associer ta branche locale à la branche distante(le rôle de -u)

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git push -u origin main
Username for 'https://github.com': hich1234
Password for 'https://hich1234@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 319 bytes | 3.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Hichem-Khelil/TP-Git.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Création d'une copie locale du repo



TP-Git

Private

Watch 0



main



1 Branch



0 Tags



Go to file



Add file



Code



Hichem-Khelil add java project

f5374ed · 35 minutes ago



1 Commit



HelloWorld.java

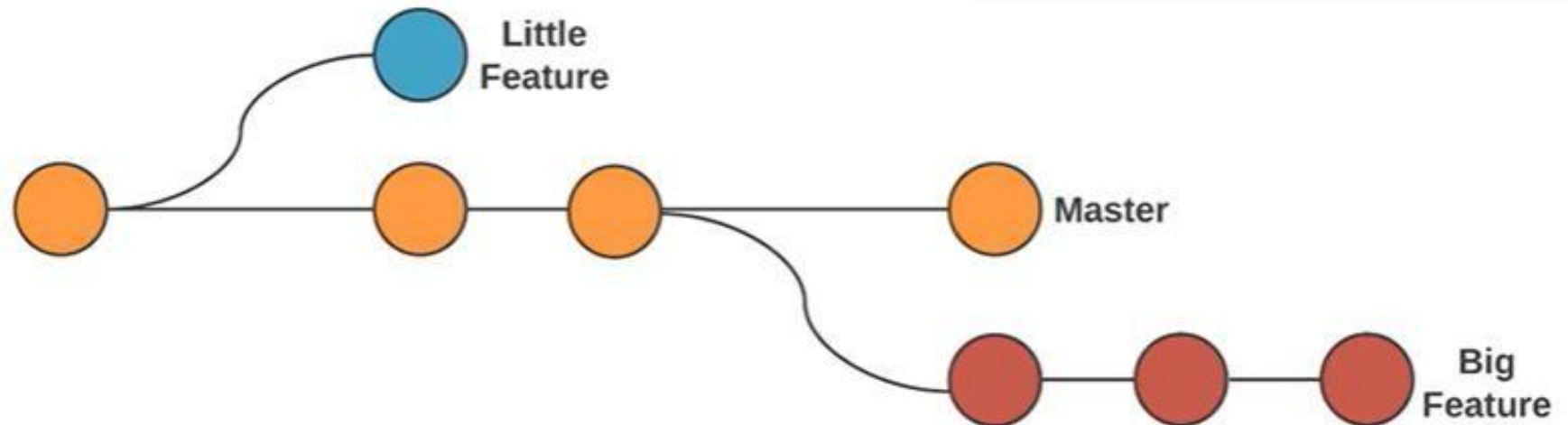
add java project

35 minutes ago

5. Les branches et la gestion de conflits

Le branchement - principe

- Vous pouvez créer plusieurs branches pour travailler sur différentes fonctionnalités ou correctifs en même temps. La commande `git branch <nom_de_la_branche>` crée une nouvelle branche à partir du dernier commit.
- Chaque branche évolue de manière indépendante, avec son propre historique de commits.
- Cela évite les conflits potentiels entre les développeurs qui travaillent sur différentes parties du projet.



La création et le changement de branches

- La plupart des systèmes de gestion de versions offrent la possibilité de gérer des branches. Lorsqu'on crée une nouvelle branche, on se sépare de la ligne principale de développement pour pouvoir travailler indépendamment, sans affecter cette ligne principale.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch
* main
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch feature1
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch
feature1
* main
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

La création et le changement de branches

- **git branch <nom de la branche>**: créer une nouvelle branche.
- **git checkout <nom de la branche>**: basculer vers la nouvelle branche.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch
feature1
* main
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git checkout feature1
Switched to branch 'feature1'
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch
* feature1
main
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

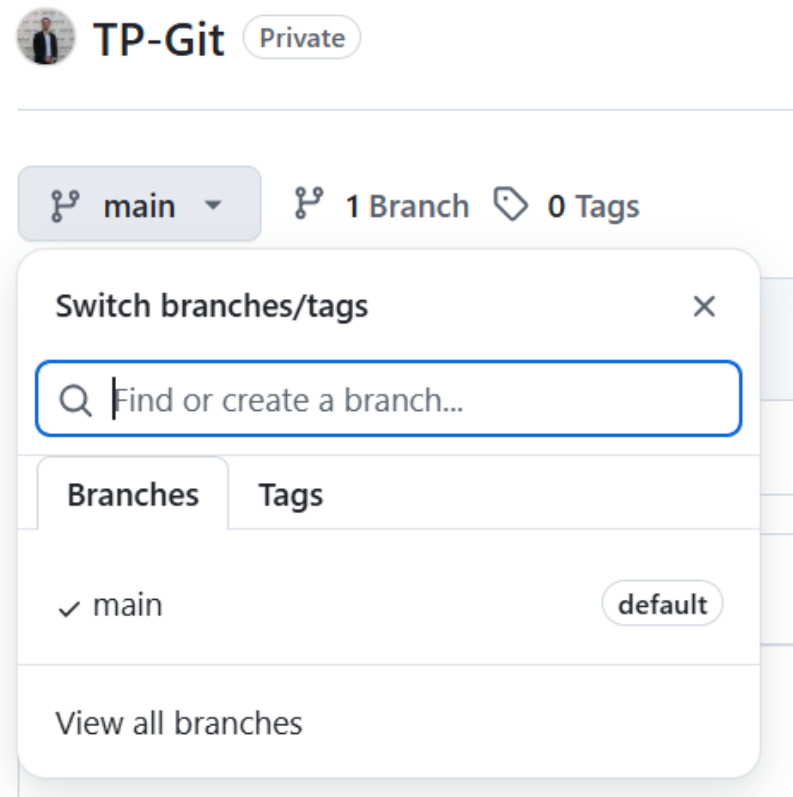
La création et le changement de branches

- **git checkout -b <nom de la branche>**: créer et basculer vers la nouvelle branche.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git checkout -b feature2
Switched to a new branch 'feature2'
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git branch
  feature1
* feature2
  main
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```


La création et le changement de branches

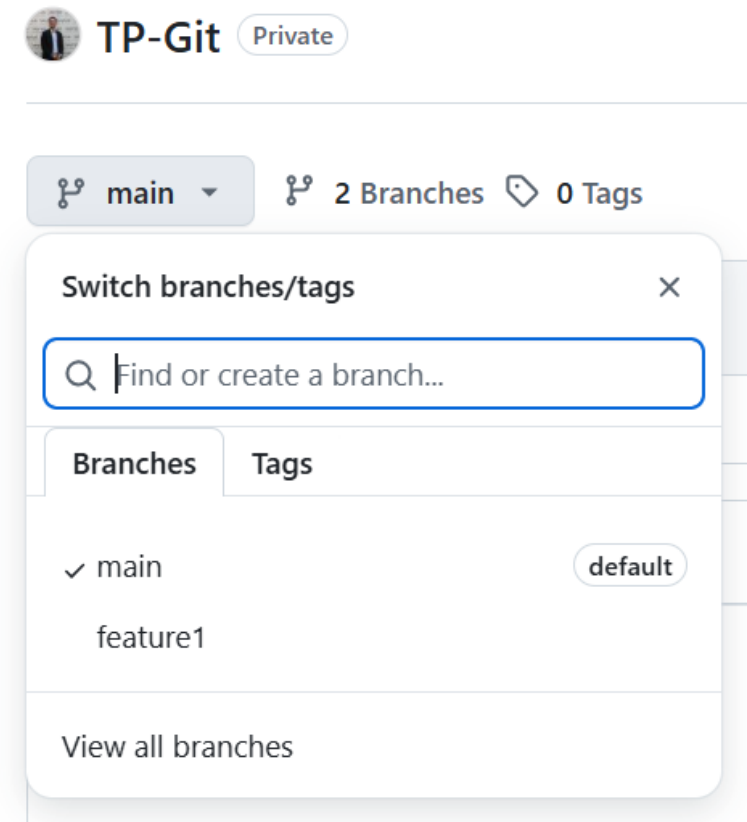
- **git push -u origin feature1**: créer la branche feature1 dans le repo distant.



La création et le changement de branches

- **git push -u origin feature1**: créer la branche feature1 dans le repo distant.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git push -u origin feature1
Username for 'https://github.com': hich1234
Password for 'https://hich1234@github.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'feature1' on GitHub by visiting:
remote:   https://github.com/Hichem-Khelil/TP-Git/pull/new/feature1
remote:
To https://github.com/Hichem-Khelil/TP-Git.git
 * [new branch]      feature1 -> feature1
Branch 'feature1' set up to track remote branch 'feature1' from 'origin'.
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```



La création et le changement de branches

- on va créer une nouvelle commit sur la branche feature1.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ vi HelloWorld.java
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ cat HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World this is a commit test");
    }
}
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git status
On branch feature1
Your branch is up to date with 'origin/feature1'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   HelloWorld.java

no changes added to commit (use "git add" and/or "git commit -a")
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git add .
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git commit -m "modification du code"
[feature1 65afd4f] modification du code
 1 file changed, 1 insertion(+), 1 deletion(-)
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git push origin feature1
Username for 'https://github.com': hich1234
Password for 'https://hich1234@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 310 bytes | 31.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Hichem-Khelil/TP-Git.git
   f5374ed..65afd4f  feature1 -> feature1
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

La création et le changement de branches

- On va maintenant basculer vers la branche principale « main » pour intégrer les modifications faites par la branche « feature1 »

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git merge feature1
Updating f5374ed..65afd4f
Fast-forward
 HelloWorld.java | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

- **git merge feature1:** intégrer les changements de la branche feature1 dans la branche main.

La Gestion des conflits

- Les conflits apparaissent généralement lorsque deux développeurs modifient les mêmes lignes d'un fichier ou lorsqu'un fichier est supprimé par un développeur alors qu'un autre le modifie. Dans ces situations, Git ne peut pas choisir automatiquement quelle version garder. Seul le développeur qui effectue le merge est affecté par le conflit, les autres membres de l'équipe ne le voient pas. Git signalera le fichier en conflit et interrompra le merge, laissant au développeur la responsabilité de résoudre le problème.

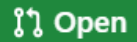
La Gestion des conflits

- On a effectué une modification sur la première ligne du fichier test1.txt en étant sur la branche feature1 et aussi feature2 .

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git merge feature1
Updating 6c61206..986990d
Fast-forward
 test1.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git merge feature2
Auto-merging test1.txt
CONFLICT (content): Merge conflict in test1.txt
Automatic merge failed; fix conflicts and then commit the result.
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

La Gestion des conflits

modifier le fichier texte #2



Hichem-Khelil wants to merge 1 commit into `main` from `feature1`



Conversation 0



Commits 1



Checks 0



Files changed 1



Hichem-Khelil commented now



No description provided.



modifier le fichier texte

986990d



This branch has conflicts that must be resolved

Use the [web editor](#) or the command line to resolve conflicts before continuing.



test1.txt

Resolve conflicts

Merge pull request



You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

La Gestion des conflits

modifier le fichier texte #2

Resolving conflicts between `feature1` and `main` and committing changes → `feature1`

1 conflicting file	test1.txt 1 conflict
 test1.txt test1.txt	Accept current change Accept incoming change Accept both changes <pre>1 <<<<<< feature1 (Current change) 2 test 3 ===== 4 test12 5 >>>>>> main (Incoming change) 6</pre>

6. Le Pull Request

Le Pull Request

- C'est une demande de fusion d'une branche dans une autre (souvent dans main) permettant aux autres développeurs de vérifier, commenter et valider les changements avant qu'ils ne soient intégrés.
- On va maintenant créer une nouvelle commit depuis la branche « feature3 » en ajoutant un nouveau fichier test2.txt.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git checkout -b feature3
Switched to a new branch 'feature3'
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ vi test2.txt
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git add .
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git commit -m "nouveau fichier"
[feature3 a2c1cb2] nouveau fichier
 2 files changed, 5 insertions(+)
 create mode 100644 test2.txt
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git push origin feature3
```



TP-Git

Private

Watch

 feature3 had recent pushes 1 minute ago

Compare & pull request

Le Pull Request

- On va maintenant créer un Pull Request depuis notre repository git.

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff co](#)

 base: main  compare: feature3  **✓ Able to merge.** These branches can be automatically merged.

**Add a title**

nouveau fichier

Add a description

WritePreview

H B I           

Add your description here...

 Markdown is supported  Paste, drop, or click to add files

Create pull request

Le Pull Request

- Une notification de Pull Request apparait.

[Code](#) [Issues](#) [Pull requests 1](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

nouveau fichier #3

[Open](#) [Hichem-Khelil](#) wants to merge 1 commit into [main](#) from [feature3](#)

[Conversation 0](#) [Commits 1](#) [Checks 0](#) [Files changed 2](#)

[Commits on Dec 1, 2025](#)

nouveau fichier

 Hichem-Khelil committed 6 minutes ago

Le Pull Request

- Aucun conflit n'est détecté, on peut faire un merge de ces modifications sur la branche main.

nouveau fichier #3

 **Open**

Hichem-Khelil wants to merge 1 commit into `main` from `feature3` 

Conversation 0

Commits 1

Checks 0

Files changed 2



Hichem-Khelil commented 3 minutes ago

...

No description provided.



  nouveau fichier

a2c1cb2



 **No conflicts with base branch**

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)



Pull request successfully merged and closed

You're all set — the `feature3` branch can be safely deleted.

Delete branch

7. Commandes git avancées

Autres commandes git

- **git log --oneline:** voir l'historique de tous les commits effectués sur un repo dans une branche spécifique .

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git log --oneline
a2c1cb2 (HEAD -> feature3, origin/feature3) nouveau fichier
e80b129 (origin/feature2, feature2) autre modification
6c61206 (origin/main) ajouter un fichier test
65afd4f modification du code
f5374ed add java project
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Autres commandes git

- **git checkout <id_commit>**: pointer à un commit spécifique (en read only). Si vous voulez effectuer des changements vous devez créer une nouvelle branche (sinon ils ne vont être associés à aucune branche et seront perdus).

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git checkout f5374ed
Note: switching to 'f5374ed'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at f5374ed add java project
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```


Autres commandes git

- **git revert <id_commit>**: annuler les changements d'un commit spécifique en créant un nouveau commit.

```
Revert "update"

This reverts commit b2656d7ae6f4f27b845c16ff7081de193d4b1595.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch feature4
# Changes to be committed:
#   modified:   test1.txt
#   deleted:    test3.txt
#
```

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git log --oneline
b2656d7 (HEAD -> feature4) update
a2c1cb2 (origin/feature3, feature3) nouveau fichier
e80b129 (origin/feature2, feature2) autre modification
6c61206 (origin/main) ajouter un fichier test
65afd4f modification du code
f5374ed add java project
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git revert b2656d7
[feature4 50505b3] Revert "update"
 2 files changed, 4 deletions(-)
 delete mode 100644 test3.txt
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Autres commandes git

- **git revert <id_commit>**: un nouveau commit a été crée pour l'action revert.

```
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$ git log --oneline
50505b3 (HEAD -> feature4) Revert "update"
b2656d7 update
a2c1cb2 (origin/feature3, feature3) nouveau fichier
e80b129 (origin/feature2, feature2) autre modification
6c61206 (origin/main) ajouter un fichier test
65afd4f modification du code
f5374ed add java project
hich-123@DESKTOP-M2CR0A6:/mnt/c/devops-module/TP-Git$
```

Travail à faire

- TP1: Git/Github: Document Word sera partagé.